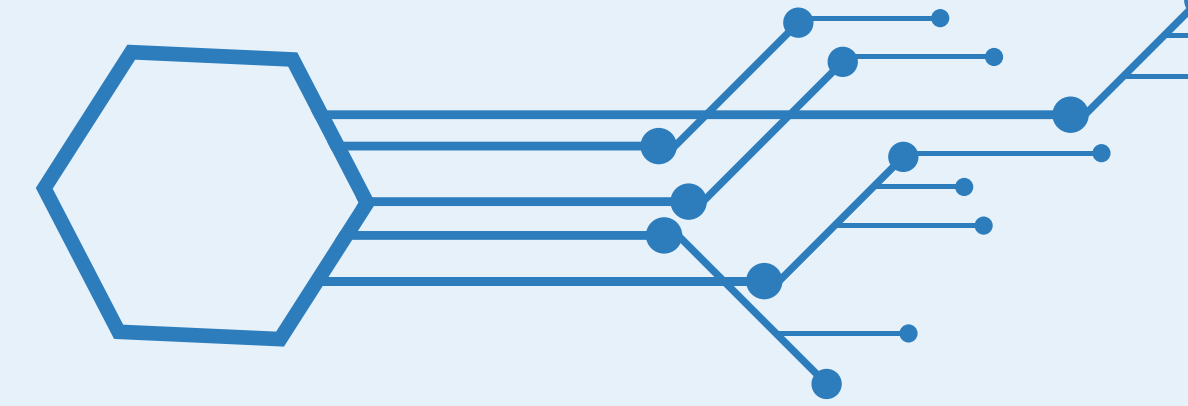




Visible Ingredient VQA from Food Images

Yemek fotoğrafından görünen malzemeleri tahmin edip ingredient sorularına Yes / Maybe / No cevabı veren mini VQA sistemi

EfficientNet-B0

Model

FoodSeg103

Dataset

103 labels

Output

Yes / Maybe / No

Decision



Problem: Sadece Yemeği Sınıflandırmak Yetmiyor

Motivasyon

Klasik yaklaşım

Image → “pizza” / “burger” / “salad”
Tek etiketli sınıflandırma

Bizim sorumuz

Fotoğrafta belirli bir malzeme var mı?
“cheese?”, “tomato?”, “rice?”

Neden daha zor?

Bir yemekte birden fazla doğru cevap var;
bazıları küçük, örtülü veya arka planda.

Amaç: Yemek fotoğrafından görünen ingredient olasılıklarını üretmek.

Sonra bu olasılıkları kullanıcı sorusuna dönüştürmek: Image + Query → Answer.

Bu yüzden problemi multi-label learning + basit ingredient-based VQA olarak kurduk.



Projeyi Nasıl Konumlandırıdık?

1. Food classification değil

Tek yemek etiketi yerine birden fazla visible ingredient tahmin ettik.

2. Segmentation datasını dönüştürdük

Pixel mask → image-level multi-label target.

3. Kullanıcı sorusu ekledik

Malzeme adı / soru → Yes / Maybe / No + confidence.

Diğer Klasik Projelerden Farkı

“Bu burger mi?” sorusu yerine “Bu görselde lettuce var mı?” diye sorulabilir hale geldi.

Model çıktısını sadece skor tablosu olarak bırakmadık; küçük bir arayüzde ingredient query cevaplayan demo yaptık.

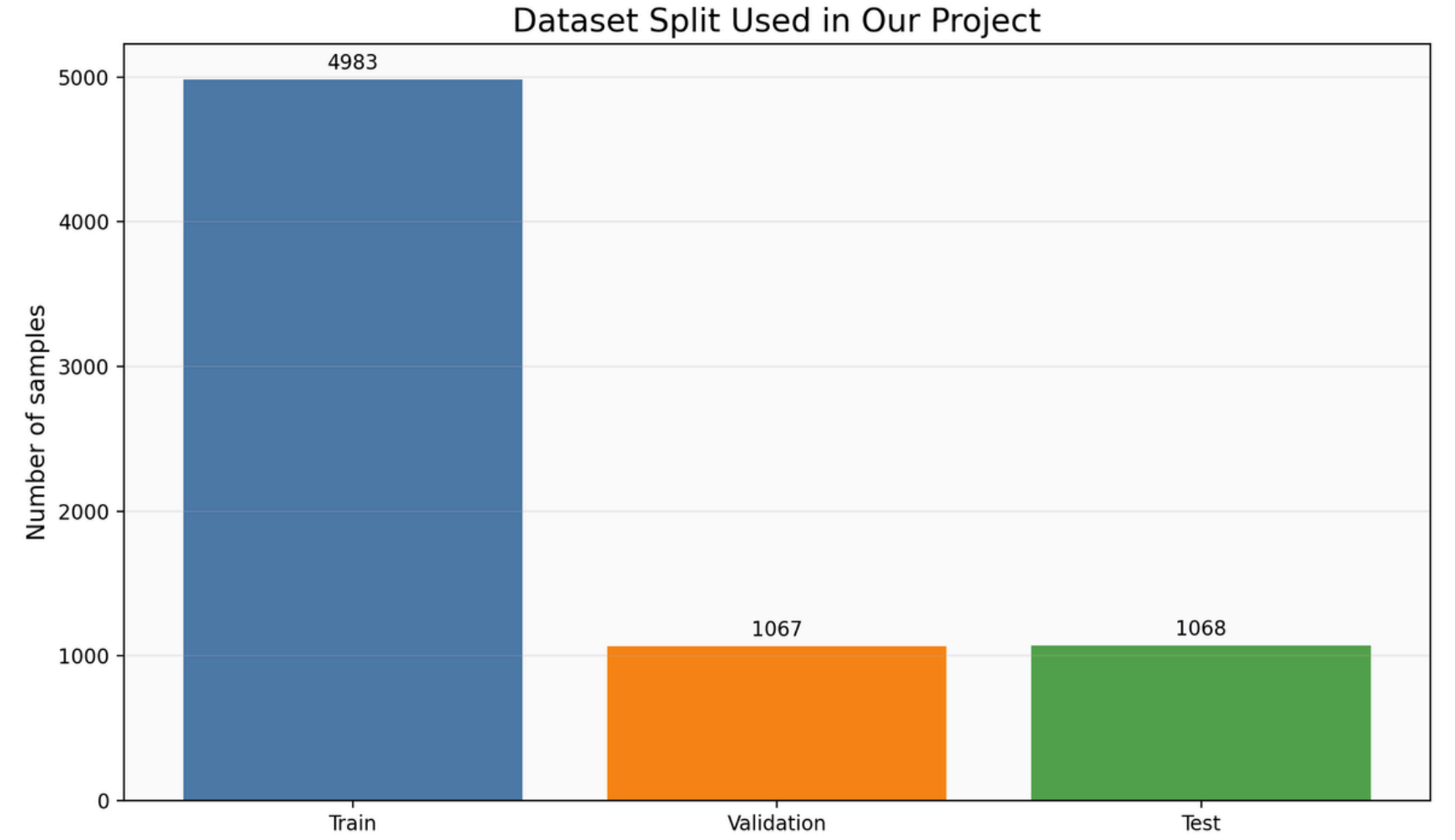
Threshold’u rastgele seçmedik; validation performansına göre optimize ettik.

Dataset: FoodSeg103

FoodSeg103: yemek görüntüleri için ingredient segmentation dataset'i.

Her örnekte: image, segmentation mask, classes_on_image ve id alanları var.

Biz maskleri direkt segmentation eğitmek için değil, image-level multi-label target üretmek için kullandık.



4,983

Train images

1,067

Validation images

1,068

Test images

103

Ingredient classes

Veri kararı

Original validation split'i ikiye bölüp validation + test olarak kullandık. Böylece model seçimi ve final test ayrıldı.

Data preparation: mask → multi-hot label



Mask içinde görülen class id'lerini classes_on_image üzerinden aldık.

Background class id = 0 olduğu için çıkardık.

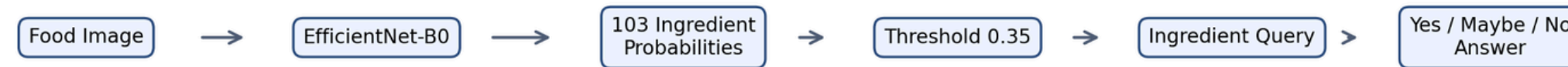
Her image için 103 uzunlukta multi-hot vektör ürettik.

Örnek: [0, 48, 66, 90] → [48, 66, 90] → 103 boyutlu label vektörü.

Neden önemli?

Bu dönüşüm sayesinde segmentation dataset'ini multi-label ingredient recognition için kullanabildik.

Model architecture



Backbone

EfficientNet-B0
food image → visual features

Output head

103 logits
1 logit per ingredient

Activation

Sigmoid
independent probabilities

- Softmax kullanmadık; çünkü tek bir doğru sınıf yok.
- BCEWithLogitsLoss kullandık; her ingredient için “var/yok” hatasını ölçüyor.
- Model output’u: cheese=0.12, tomato=0.76, rice=0.04 gibi probability vektörü.

Training setup

Hardware

Local training
RTX 3050 Laptop GPU
CUDA enabled

Training config

Full train set
10 epochs
batch size 16
image size 224

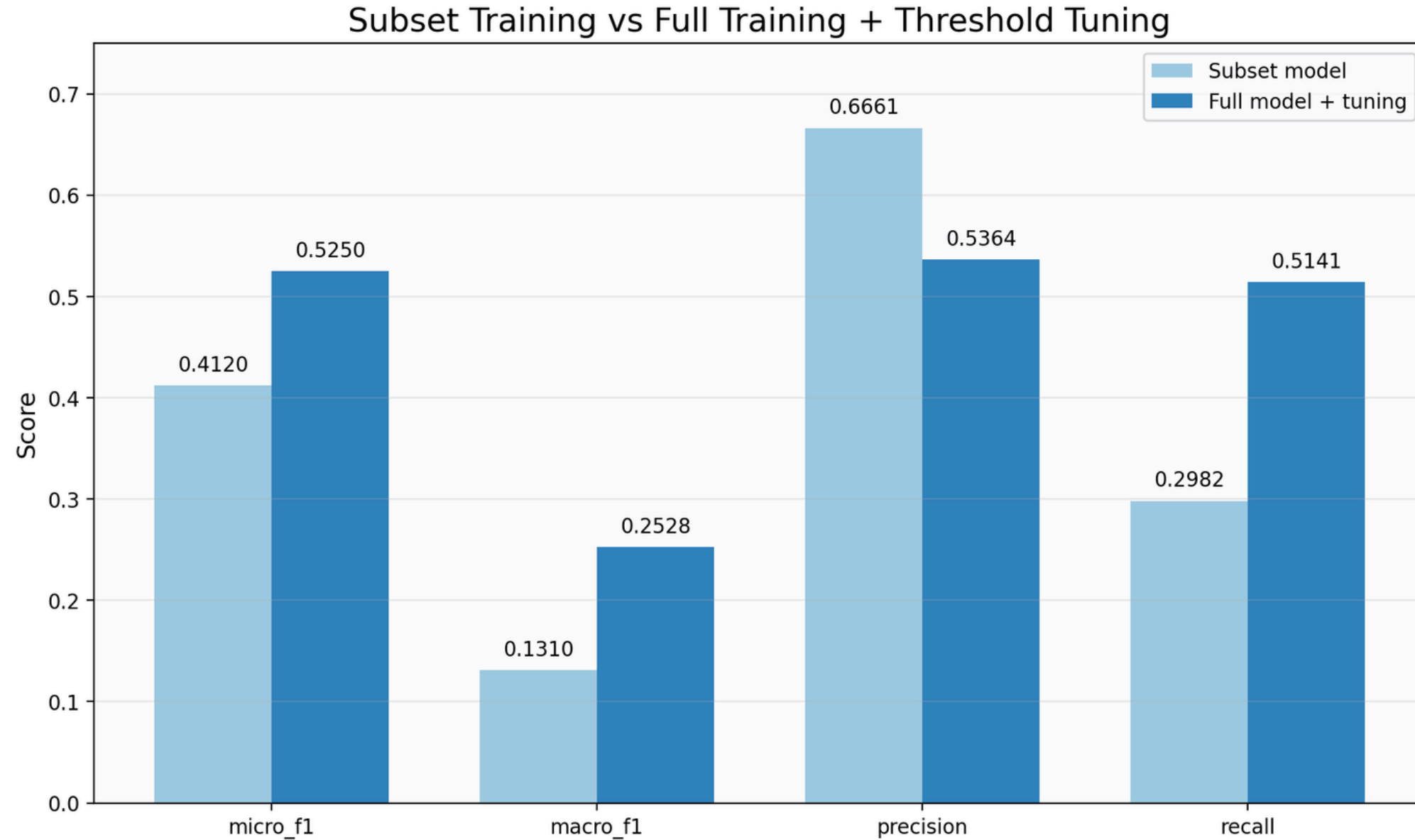
Optimization

EfficientNet-B0
BCEWithLogitsLoss
LR = 0.0003

Eğitim stratejisi

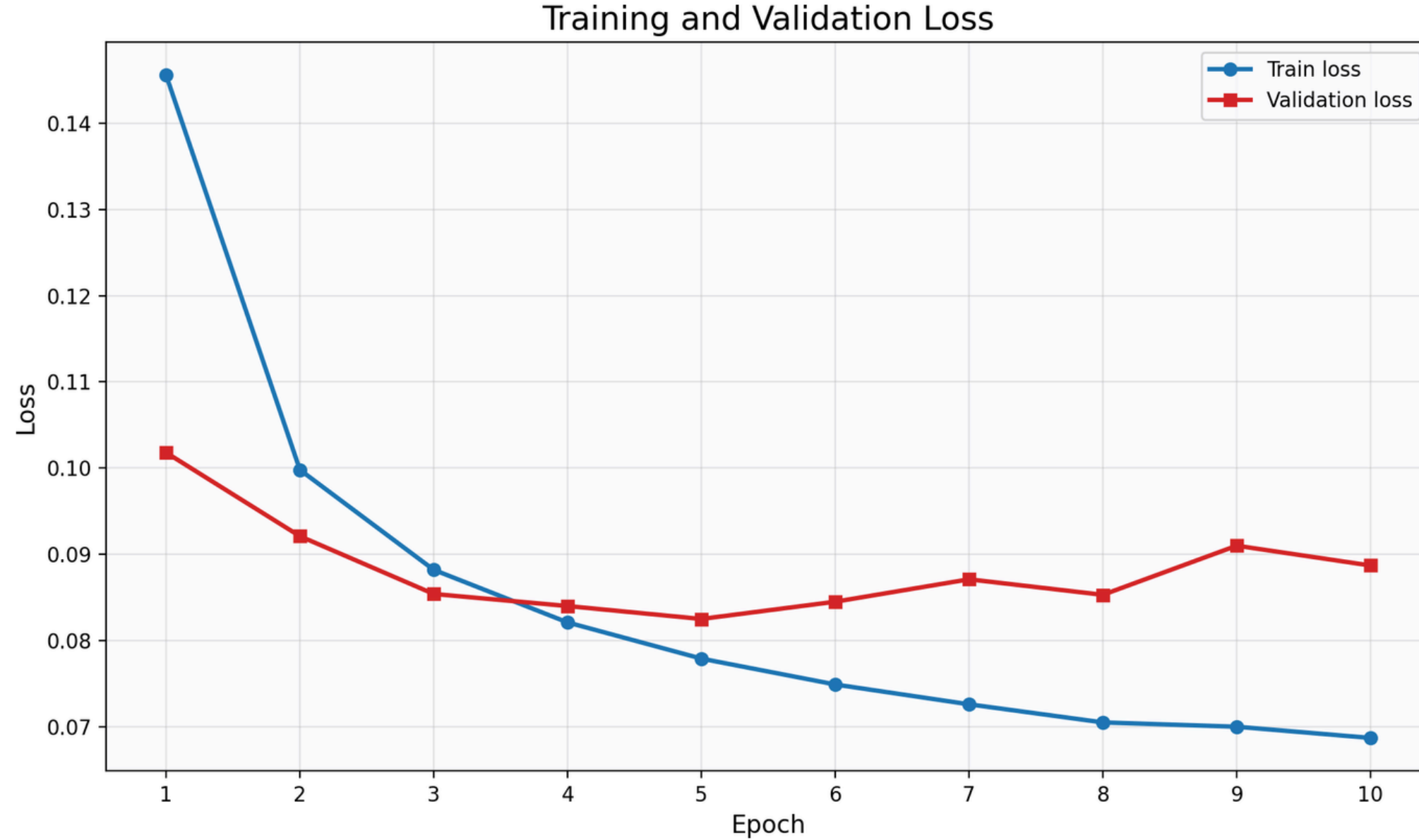
Önce küçük subset smoke test: data loader, loss ve checkpoint akışı doğru mu kontrol edildi.
Sonra tüm train set ve 10 epoch ile final eğitim yapıldı.
En iyi checkpoint validation micro F1'e göre kaydedildi.

İlk denemeden finale: Karar hikâyesi



- İlk subset model: Sistem çalıştı ama recall düşüktü; model çok temkinliydi.
- Full training: Daha fazla veri ve 10 epoch ile micro F1 arttı.
- Threshold tuning: 0.5 yerine validation'da en iyi dengeyi veren 0.35 seçildi.
- Sonuç: recall büyük ölçüde yükseldi; precision biraz düştü ama F1 dengelendi.

Training behavior: loss



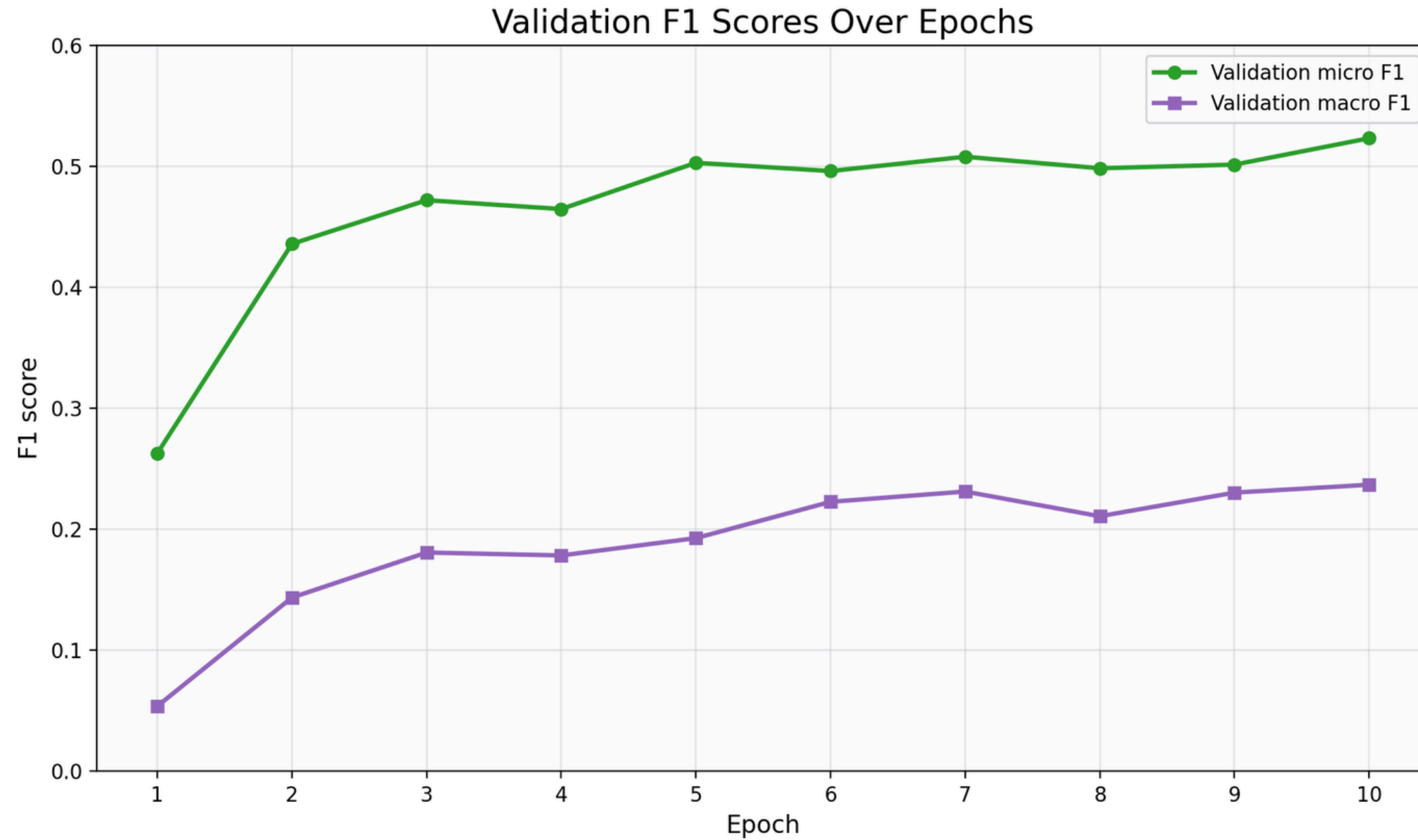
Ne görüyoruz?

Train loss sürekli düşüyor. Validation loss ilk epochlarda düşüyor, sonra hafif dalgalanıyor.

Yorum

Model öğreniyor; sonlara doğru küçük overfitting/dalgalanma var. Checkpoint seçiminde validation micro F1 kullandık.

Training behavior: validation F1



Micro F1

Genel ingredient tahmin performansı.
0.2628 → 0.5231

Macro F1

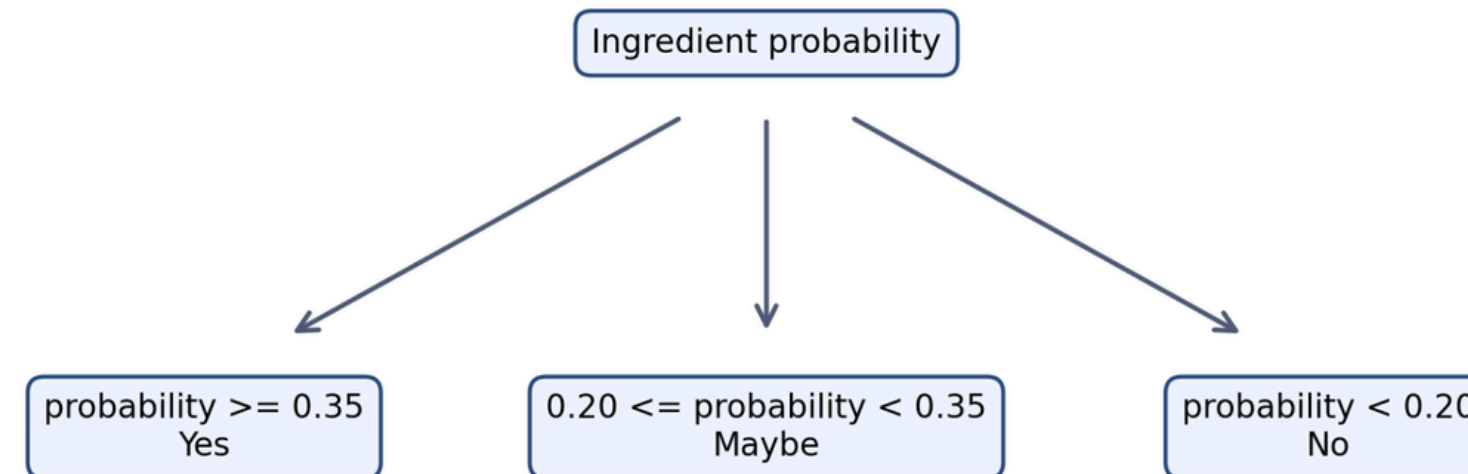
Nadir sınıfları da eşit ağırlıkla değerlendirir.
Düşük kalması class imbalance işareti.

Öğrenme

Model sık görülen ingredientleri daha iyi öğreniyor; Nadir sınıflar daha zor.

Problem: probability → karar dönüşümü

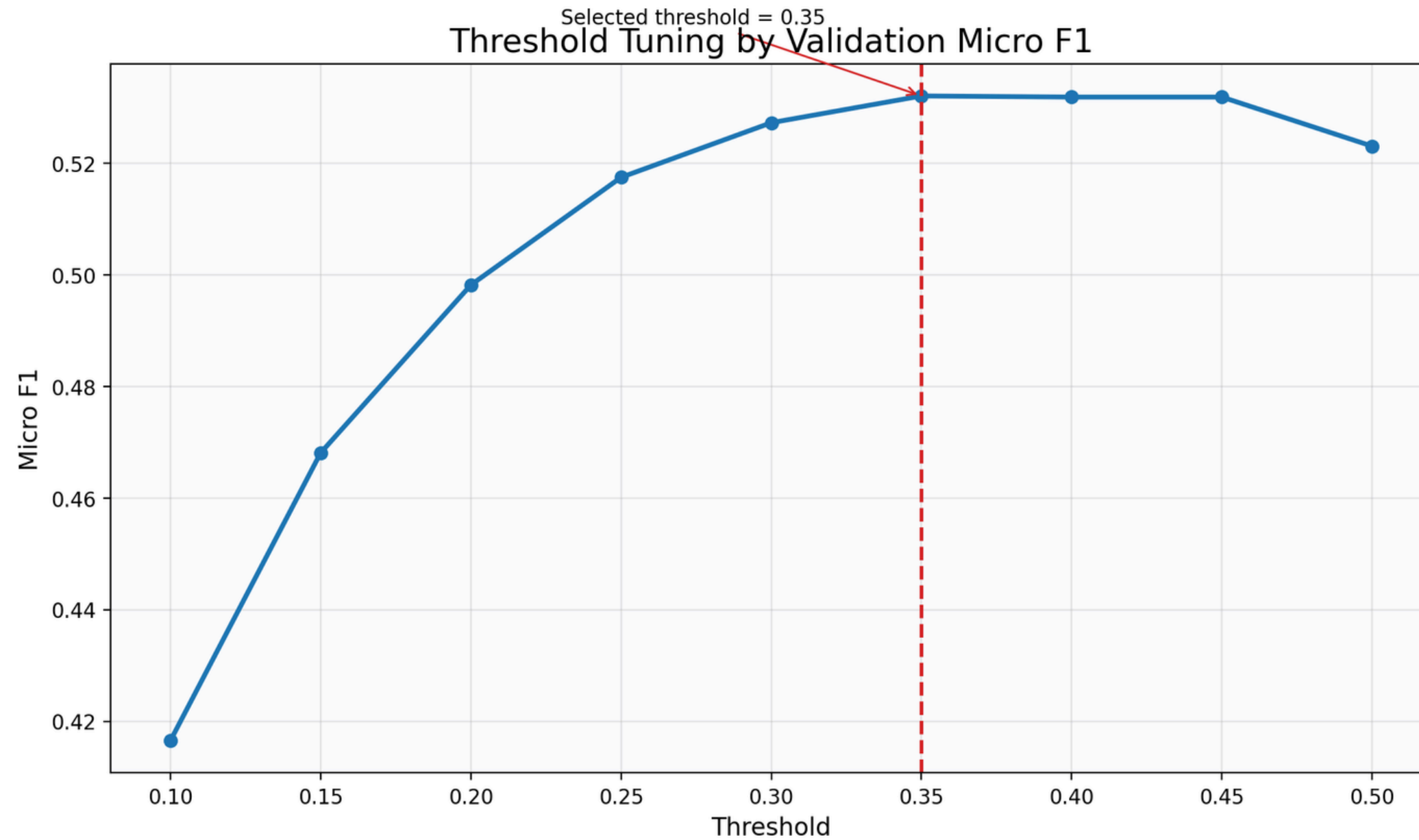
Model her ingredient için probability veriyor; Kullanıcıya cevap vermek için threshold gerekiyor. Varsayılan 0.5 threshold modelin çok fazla “No” demesine neden oldu. Bu yüzden threshold’u validation set üzerinde sistematik olarak seçtik.



Final UI kararı

$p \geq 0.35 \rightarrow \text{Yes}$ | $0.20 \leq p < 0.35 \rightarrow \text{Maybe}$ | $p < 0.20 \rightarrow \text{No}$

Threshold tuning: neden 0.35?



- Validation micro F1 en yüksek değere 0.35 civarında ulaştı.
- 0.50 çok katı kaldı; bazı gerçek ingredientler kaçırıldı.
- 0.10–0.20 çok gevşek kaldı; **FALSE-POSITIVE** sayısı arttı.

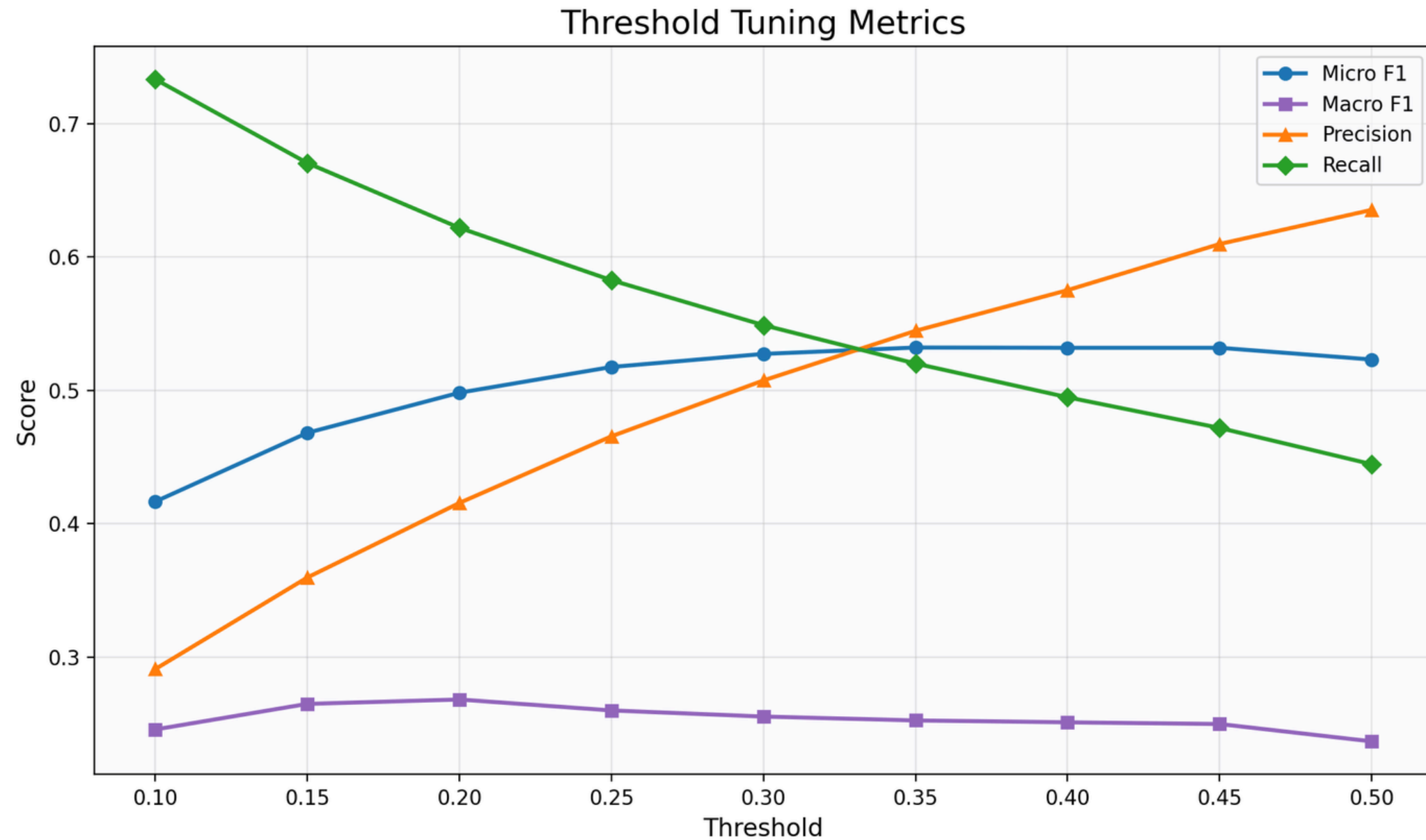
0.35

Selected threshold

0.5321

Validation micro F1

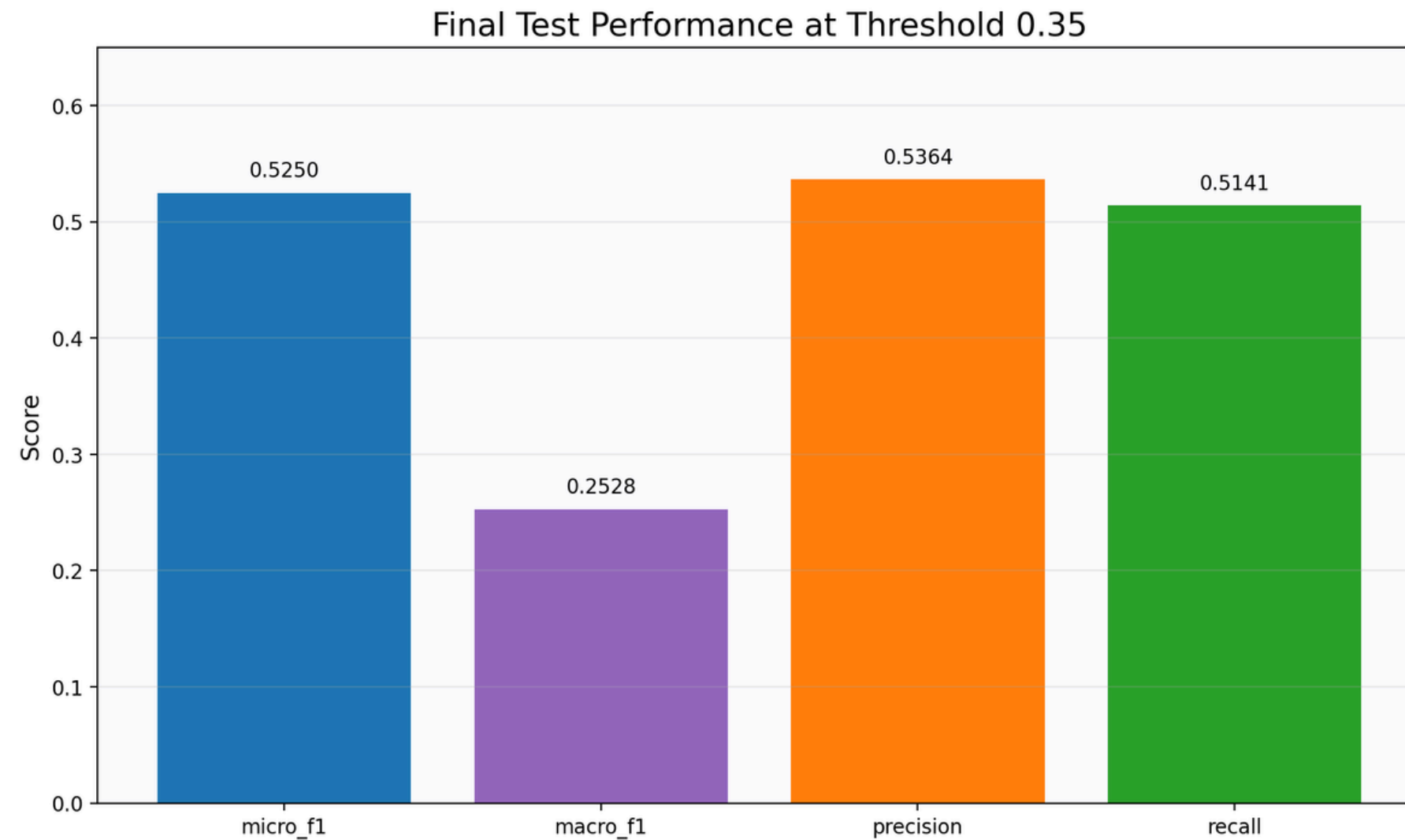
Threshold tuning: precision–recall trade-off



Threshold düşükken recall yüksek:
model çok “Yes” diyor.

Threshold yüksekken precision
yüksek: model daha seçici oluyor.
0.35: precision ve recall arasında
daha dengeli nokta.

Final quantitative results



0.5250

Test micro F1

0.2528

Test macro F1

0.5364

Precision

0.5141

Recall

Micro F1: genel performans için en uygun özet metrik.

Macro F1 düşük: nadir ingredient sınıflarında performans zayıf.

Precision/recall dengesi threshold tuning sonrası belirgin iyileşti.

Real-world test: başarılı örnek



Hamburger

Modelin güçlü olduğu durum:
malzemeler büyük, ayırık ve görsel olarak belirgin.

- Top predictions: bread 0.9901, lettuce 0.9677, steak 0.8935, tomato 0.7637.
- Bu örnekte model hem ana yapıyı hem de visible ingredientleri iyi yakaladı.
- Genel Batı yemekleri ve net görseller dataset dağılımına daha yakın görünüyor.

Real-world test: kısmen başarılı örnek



Korean bowl

Egg çok iyi yakalandı; bazı sebzeler düşük confidence kaldı.

- egg: 0.9799 → Yes
- noodles: 0.8206 → güçlü tahmin
- carrot: 0.2487 → threshold altında; Maybe/uncertain olarak yorumlanabilir.
- Ders: Doğru ingredient bazen “No” değil, düşük güvenli tahmin olabilir.

Real-world test: zorlandığı durumlar



Pizza

Pizza class doğru; pepper/cheese kaçtı, mantar karışıklığı.



Kebap

Tomato/steak doğru; yeşil biber asparagus ile karıştı.



Tavuk pilav

Arka plan domates/yeşillik modelin dikkatini dağıttı; chicken/rice kaçtı.

- Bu örnekler modelin test setinde ölçülen performansının gerçek-world görüntülere birebir taşınmadığını gösterdi.
- Ana sebepler: domain shift, arka plan karışıklığı, küçük/örtülü malzemeler ve image-level model kullanmamız.

Qualitative analysis: Gerçek görsellerden çıkardığımız dersler

Görsel Türü (Image type)	Doğru Davranış (Correct behavior)	Hata (Mistake)	Çıkarılan Ders (Lesson)
Hamburger	Bread, lettuce, steak, tomato detected well	Minor omissions	General Western foods work better
Korean food	Egg detected	Carrot uncertain or missed	Some visible ingredients remain low-confidence
Pizza	Pizza class detected	Pepper or cheese missed, mushrooms confused	Ingredient-level detail is harder than dish-level cues
Kebab	Tomato and steak detected	Pepper confused as asparagus	Domain shift affects fine-grained labels
Chicken rice	Some greens detected	Missed chicken and rice	Background cues can dominate the prediction
Salad	Tomato present visually	Tomato below yes threshold	Thresholding changes the user-facing answer

Ana çıkarım

Model benchmark içinde ölçülebilir başarı verdi; internet/Türk yemeklerinde domain shift ve background etkisi belirginleşti.

Limitations: Neden hatalar oldu?

1. Domain shift

FoodSeg103 dağılımı ile internet/Türk yemekleri aynı değil.

2. Image-level model

Model pixel-level segmentation yapmıyor; küçük ingredientleri kaçırabiliyor.

3. Background confusion

Tavuk pilav örneğinde arka plan objeleri tahmini etkiledi.

4. Label coverage

cheese butter, chicken duck gibi birleşik sınıflar kullanıcı sorularını zorlaştırıyor.

5. Hidden ingredients

Sos, baharat, sarımsak gibi ingredientler görüntüden doğrudan anlaşılamayabilir.

6. Class imbalance

Macro F1 düşük; nadir sınıflar sık görülenler kadar iyi öğrenilmedi.

Bu hatalar “başarısızlık” değil; modelin neyi öğrenip neyi öğrenemediğini anlamamızı sağladı.

Final bulgular

Presentation Summary

What worked

Hamburger ingredient detection,
egg detection,
and pizza class recognition

What improved

Full training + threshold tuning
improved micro F1
from 0.412 to 0.525

What failed

Domain shift, Turkish dishes,
small or hidden ingredients,
and background confusion

- En güçlü taraf: görüntüden çoklu ingredient tahmini + ingredient query'ye cevap verme akışı uçtan uca çalıştı.
- En net iyileştirme: full training + threshold tuning ile micro F1 0.412'den 0.525'e çıktı.
- En önemli limit: model dataset dışı yemeklerde, özellikle Türk yemekleri ve karışık sahnelerde zayıflıyor.

Conclusion

Ne yaptık?

Segmentation dataset'ini multi-label ingredient recognition datasına çevirdik ve EfficientNet-B0 eğittik.

Ne öğrendik?

Model eğitmek kadar threshold, veri dağılımı ve gerçek-world hata analizi de önemli.

Final sistem

Food image + ingredient query → Yes / Maybe / No + confidence + top predictions.

Ana sonuç

Test micro F1 = 0.525; gerçek görsellerde başarı örneğe ve domain'e bağlı.



Ek görseller / sunumda eklenebilecek materyaller

Gradio demo screenshot: app arayüzünün çalıştığını göstermek için Slide 15'e eklenebilir.

Dataset örneği: FoodSeg103 image + mask görseli varsa Slide 5'e eklenebilir.

Canlı demo: hamburger.jpg ile tomato/bread/lettuce sorguları; Korean food ile egg sorgusu.

Failure demo: salad tomato → Maybe, tavuk pilav → background confusion örneği.

Figure index: training, threshold ve qualitative analysis grafiklerinin kaynak listesi.

Buraya Gradio ekran görüntüsü veya canlı demo QR/link eklenebilir

Buraya FoodSeg103 image + mask örneği eklenebilir